Azure ETL for Beginners: Learner Setup Guide

Oct 24, 2026 · @Tiger

In about 90 minutes you will build a real ETL pipeline on Azure that takes a messy orders file, cleans it, and loads 4 tidy rows into a database. No Azure experience is needed; every click is written out.

What you will build

Think of a pizza shop. A delivery of messy ingredients arrives in the **store room**. The **kitchen** washes and sorts everything. Only good ingredients go onto the **display shelf**.

Shop picture	Azure service	What it does	Name we will use
A folder that holds everything	Resource group	Keeps all your pieces together so you can delete them in one go	`rg-etl-training`
Store room	Storage account + container	Keeps the messy CSV file exactly as it arrived	`stetltraining` + `orders`
Display shelf	Azure SQL Database	Stores only the clean rows in a table	`training` + `dbo.orders_clean`
Kitchen manager	Data Factory	Moves and cleans the data	`adf-etl-training`
Address book	Linked services	Saved connections to the store room and the shelf	`ls_blob_orders`, `ls_sql_orders`
Box labels	Datasets	Say which file or table, and what it looks like	`ds_orders_csv`, `ds_orders_sql`
Washing machine	Data flow	Cleans the data step by step	`df_clean_orders`
To-do card	Pipeline	Tells the machine when to run	`pl_orders_etl`

How it fits the ETL idea

- **Extract:** the data flow reads the CSV from the storage account.
- **Transform:** the data flow trims text, removes bad rows, removes duplicates and adds a total column.
- **Load:** the data flow writes the clean rows into the SQL table.

The cleaning happens in the middle, in Data Factory, before the data reaches the database. That is what makes this ETL and not ELT.

Before you start

You need:

- A Microsoft Azure account with an active subscription. A free account works.
- A laptop with a modern browser such as Edge or Chrome.
- Permission to create resources in your subscription (the Owner or Contributor role).
- About 90 minutes and a stable internet connection.

Good habits for the whole lab:

- Use one region for everything (this guide uses Central India; any region works if you stay consistent).
- Names matter. Copy the names from the table in the overview, but add your own initials where a name must be unique (storage account, SQL server, Data Factory).
- After each part, stop and check the result before moving on.

Names that must be unique across all of Azure:

Resource	Rules	Example
Storage account	3 to 24 characters, lowercase letters and numbers only	`stetltrainingabc`
SQL server	Lowercase letters, numbers and hyphens	`sql-etl-abc`
Data Factory	3 to 63 characters, letters, numbers and hyphens	`adf-etl-abc`

The sample file. Save the text below in Notepad as `orders_2026-10-24.csv` (choose *Save as type: All files* so it does not become `.txt`). It is deliberately messy: order 1002 appears twice, order 1003 has no customer, order 1004 has a negative quantity, and the

region names have mixed capitals and spaces.

```
order_id,customer_id,order_date,product,quantity,unit_price,region,status
1001,C001,2026-10-20,Laptop,1,55000,south,Delivered
1002,C002,2026-10-20,Mouse,2,500, North ,delivered
1002,C002,2026-10-20,Mouse,2,500, North ,delivered
1003,,2026-10-21,Keyboard,1,1500,East,Shipped
1004,C004,2026-10-21,Monitor,-1,9000,WEST,Returned
1005,C005,2026-10-22,Laptop,2,55000,South,Delivered
1006,C006,2026-10-23,Headset,3,2500,north,Shipped
```

The goal. After cleaning, only 4 of the 7 rows should reach the database: orders 1001, 1002, 1005 and 1006. Keep that number in mind; it is how you will know you got it right.

Part 1: Create the resource group (your project folder)

A resource group is a folder that holds everything for this lab. When you finish, deleting this one folder removes every resource and stops all charges.

1. Go to portal.azure.com and sign in.
2. In the search bar at the top, type **Resource groups** and open it.
3. Click **+ Create**.
4. Choose your **Subscription**, type the name `rg-etl-training`, and pick **Region**: Central India.
5. Click **Review + create**, then **Create**.

Check: `rg-etl-training` appears in your Resource groups list.

Part 2: Create the storage account (the store room)

1. In the top search bar type **Storage accounts** and open it. Click **+ Create**.
2. On the **Basics** tab fill in:
   - Resource group: `rg-etl-training`
   - Storage account name: your unique name, for example `stetltrainingabc`
   - Region: the same region as before
   - Performance: **Standard**
   - Redundancy: **Locally-redundant storage (LRS)**, the cheapest option

3. Leave every other tab as it is. Click **Review + create**, then **Create**. Wait for "Your deployment is complete", then click **Go to resource**.
4. In the left menu open **Data storage** → **Containers**.
5. Click **+ Container**, type the name `orders` (lowercase), and click **Create**.
6. Click the new `orders` container to open it, then click **Upload**.
7. Click **Browse for files**, choose `orders_2026-10-24.csv`, and click **Upload**.

Check: inside the `orders` container you can see `orders_2026-10-24.csv`. Click it, then **Edit** to see the 7 data rows if you want to be sure the content saved correctly.

Part 3: Create the Azure SQL Database (the display shelf)

1. In the top search bar type **SQL databases** and open it. Click **+ Create**.
2. On the **Basics** tab:
   - Resource group: `rg-etl-training`
   - Database name: `training`
   - Server: click **Create new**
3. In the **Create SQL Database Server** pane:
   - Server name: your unique name, for example `sql-etl-abc`
   - Location: the same region as before
   - Authentication method: **Use Microsoft Entra-only authentication**
   - Click **Set admin**, search for your own account, select it, and click **Select**. You are now the admin.
   - Click **OK**.
4. Back on Basics, set **Workload environment** to **Development**. Under **Compute + storage** click **Configure database**, choose **Serverless**, set **Max vCores** to 1, set **Auto-pause** to 1 hour, and click **Apply**. Serverless pauses itself when idle, so it costs very little.
5. Set **Backup storage redundancy** to **Locally-redundant**.
6. Open the **Networking** tab:
   - Connectivity method: **Public endpoint**
   - Allow Azure services and resources to access this server: **Yes**
   - Add current client IP address: **Yes**
7. Click **Review + create**, then **Create**. When it finishes, click **Go to resource**.

Why Entra-only? Newer Azure SQL servers are created without a username and password. Sign-in uses your Microsoft account instead. This is more secure, and Part 4 shows how Data Factory signs in the same way.

Create the table

1. On your database page, open **Query editor (preview)** in the left menu.
2. Choose **Continue as** your account (Microsoft Entra authentication). If you see a firewall message, click the link to add your IP and try again.
3. Paste this and click **Run**:

```
CREATE TABLE dbo.orders_clean (
    order_id      INT           NOT NULL PRIMARY KEY,
    customer_id   VARCHAR(20)   NOT NULL,
    order_date    DATE          NOT NULL,
    product       VARCHAR(100)  NOT NULL,
    quantity      INT           NOT NULL,
    unit_price    DECIMAL(10,2) NOT NULL,
    region        VARCHAR(20)   NOT NULL,
    status        VARCHAR(20)   NOT NULL,
    total_amount  DECIMAL(12,2) NOT NULL
);
```

Check: the message says the query succeeded. Expand **Tables** in the left Explorer pane (use the refresh icon) and you will see `dbo.orders_clean`. The table is empty for now. That is correct.

Part 4: Create Data Factory (the kitchen manager)

Create the factory

1. In the top search bar type **Data factories** and open it. Click **+ Create**.
2. On the **Basics** tab:
   - Resource group: `rg-etl-training`
   - Name: your unique name, for example `adf-etl-abc`
   - Region: the same region as before
   - Version: **V2**
3. Open the **Git configuration** tab and tick **Configure Git later**.

4. Click **Review + create**, then **Create**. When it finishes, click **Go to resource**, then **Launch studio**. A new browser tab opens. This is Data Factory Studio, where you will spend the rest of the lab.

The studio has four icons on the left: **Home**, **Author** (the pencil, where you build things), **Monitor** (the gauge, where you watch runs), and **Manage** (the toolbox, where connections live).

Give Data Factory permission to use the SQL database

The SQL database only lets in people it knows. Data Factory has its own identity, a bit like a staff badge, and it carries the same name as your factory. You must tell SQL that this badge is allowed.

1. Go back to the Azure portal tab and open your `training` database → **Query editor (preview)**. Sign in with Microsoft Entra authentication.
2. Paste this, **replacing `adf-etl-abc` with your exact Data Factory name** in all four places, and click **Run**:

```
CREATE USER [adf-etl-abc] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [adf-etl-abc];
ALTER ROLE db_datawriter ADD MEMBER [adf-etl-abc];
ALTER ROLE db_ddladmin  ADD MEMBER [adf-etl-abc];
```

The four lines let the factory create its login, read rows, write rows, and empty the table before each load.

Check: the query succeeds. If you see "Principal could not be found", the name does not exactly match your Data Factory name.

Create the two linked services (the address book)

In Data Factory Studio click **Manage** → **Linked services** → **+ New**.

Linked service 1: the store room

1. Search **Azure Blob Storage**, select it, click **Continue**.
2. Name: `ls_blob_orders`.
3. Authentication type: **Account key**. Under *Storage account name* choose **From Azure subscription**, then select your subscription and your storage account.
4. Click **Test connection**. It should say **Connection successful**. Click **Create**.

Linked service 2: the shelf

1. Click **+ New**, search **Azure SQL Database**, select it, click **Continue**.
2. Name: `ls_sql_orders`.
3. Choose your subscription, your server and the database `training`.
4. Authentication type: **System-assigned managed identity**. This is the staff badge. Do not enter a username or password.
5. Click **Test connection**. It should say **Connection successful**. Click **Create**.

Check: the Linked services list shows two items with no errors. If the SQL test fails, see the troubleshooting table at the end.

Part 5: Create the two datasets (the box labels)

A linked service opens the door. A dataset says exactly which file or table you mean. Click **Author** (the pencil), hover over **Datasets**, click the **…** next to it, and choose **New dataset**. Do this twice.

Dataset 1: the CSV file

1. Search **Azure Blob Storage**, select it, click **Continue**.
2. Choose the format **DelimitedText**, click **Continue**.
3. Name: `ds_orders_csv`. Linked service: `ls_blob_orders`.
4. Click the folder icon next to **File path** and select `orders` → `orders_2026-10-24.csv`. If the folder icon shows nothing, type `orders` in the first box and `orders_2026-10-24.csv` in the last box instead.
5. Tick **First row as header**.
6. Set **Import schema** to **None** for now, then click **OK**.

Dataset 2: the SQL table

1. Click **New dataset** again, search **Azure SQL Database**, click **Continue**.
2. Name: `ds_orders_sql`. Linked service: `ls_sql_orders`.
3. Open the **Table name** dropdown and choose `dbo.orders_clean`. If the list is empty, tick **Enter manually**, type `dbo` in the first box and `orders_clean` in the second.
4. Click **OK**.

Check: click each dataset and use **Test connection** on the Connection tab. Both should succeed.

Two small things to know. The dots next to dataset names mean unsaved changes; they go away after you publish in Part 7. And a dataset with a red error usually means its linked service cannot connect, so fix the linked service first.

Part 6: Build the data flow (the washing machine)

The data flow is a row of stations on a canvas. Rows enter on the left, pass through each station, and leave on the right. You will build five stations plus the starting source.

Start the data flow

1. Click **Author**, hover over **Data flows**, click **...**, and choose **New data flow**. Choose **Data flow** (not Wrangling).
2. In the Properties pane on the right, name it `df_clean_orders`.
3. Turn on **Data flow debug** at the top of the canvas and click **OK**. The helper computers need 3 to 5 minutes to warm up. Keep working while you wait.

Station 1: Source (pick up the file)

1. Click **Add Source** on the canvas.
2. In the settings below, set Source type to **Dataset** and Dataset to `ds_orders_csv`.
3. Open the **Projection** tab. When debug is ready, click **Detect data types** (or **Import projection** if the column list is empty). You should see all 8 column names, with `quantity` and `unit_price` as numbers.

The source must be the CSV only. Never add the SQL dataset as a source.

Station 2: Derived column (tidy the text)

1. Click the **+** at the bottom right of the source box and choose **Derived Column**.
2. Name the stream `tidyText`.
3. Add two columns. Type each name in the left box and the expression in the right box (click the box to open the expression builder):
   - `region` = `initCap(trim(region))`
   - `status` = `initCap(trim(status))`

`trim` removes spaces, and `initCap` makes the first letter of each word a capital. So `" North "`, `north` and `WEST` become `North`, `North` and `West`.

Station 3: Filter (throw out bad rows)

1. Click **+** on the previous box and choose **Filter**. Name it `dropBadRows`.
2. In **Filter on**, enter:

```
!isNull(customer_id) && trim(customer_id) != '' && quantity > 0
```

This keeps a row only if it has a customer and a positive quantity. Orders 1003 and 1004 are dropped.

Station 4: Aggregate (remove the twins)

1. Click **+** and choose **Aggregate**. Name it `removeDuplicates`.
2. On the **Group by** tab choose the column `order_id`.
3. Open the **Aggregates** tab and click **Add column pattern** (the pattern option, not a single column). Fill in:
   - Matching condition: `name != 'order_id'`
   - Column name: `$$`
   - Expression: `first($$)`

In plain words: group the rows by order number, and for every other column keep the first value. The two copies of order 1002 become one.

Station 5: Derived column (add the total)

1. Click **+** and choose **Derived Column**. Name it `addTotal`.
2. Add one column: `total_amount` = `quantity * unit_price`.

Station 6: Sink (put it on the shelf)

1. Click **+** and choose **Sink**. Name it `loadToSql`.
2. Dataset: `ds_orders_sql`.
3. Open the **Settings** tab. Tick **Allow insert**. In **Pre SQL scripts** type:

```
TRUNCATE TABLE dbo.orders_clean
```

This empties the table before each load, so running the pipeline twice does not create duplicate rows.

Then open the **Mapping** tab and keep **Auto mapping** on. If the sink complains about `order_date`, add a column `order_date` = `toDate(order_date)` in the `addTotal` station.

Check each station

With debug on, click a station, open **Data preview**, and click **Refresh**. You should see these row counts:

After station	Rows you should see
Source	7
tidyText	7
dropBadRows	5
removeDuplicates	4
addTotal	4, with `total_amount` filled in

If the count is wrong, fix that station before moving on. Do not continue with wrong numbers.

Part 7: Create the pipeline, run it and check the result

The data flow is the machine. The pipeline is the instruction card that says "run the machine now".

Build the pipeline

1. Click **Author**, hover over **Pipelines**, click **...**, and choose **New pipeline**.
2. In the Properties pane name it `pl_orders_etl`.
3. In the Activities pane search **Data flow**, and drag **Data flow** onto the canvas.
4. Click the activity. On the **Settings** tab choose Data flow: `df_clean_orders`. Leave the compute size on **Small** and the integration runtime on the default (AutoResolve).

Run it

1. Click **Validate** at the top of the canvas. Fix anything it reports.
2. Click **Debug**. A panel opens at the bottom.
3. Open the **Output** tab and click the refresh icon every 20 to 30 seconds. The status moves from *Queued* to *In progress* to **Succeeded**. The first run takes about 3 to 6 minutes.

Check the result in SQL

Go to the Azure portal → your `training` database → **Query editor** and run:

```
SELECT * FROM dbo.orders_clean ORDER BY order_id;
```

You should see exactly these 4 rows:

order_id	customer_id	product	quantity	region	status	total_amount
1001	C001	Laptop	1	South	Delivered	55000.00
1002	C002	Mouse	2	North	Delivered	1000.00
1005	C005	Laptop	2	South	Delivered	110000.00
1006	C006	Headset	3	North	Shipped	7500.00

If you see these 4 rows, you have built a working ETL pipeline.

Save and automate it

1. Click **Publish all** at the top, then **Publish**. This saves your work permanently. Until you publish, changes exist only in your browser session.
2. To run it by itself, click **Add trigger** → **New/Edit** → **New**, choose **Schedule**, set it to repeat every 1 day, and click **OK**. The pipeline then runs every day without anyone clicking Debug.

A scheduled trigger runs the published version, so publish first.

When something goes wrong

These are the problems learners hit most often, with the message you will see and the fix.

What you see	What it means	Fix
"Azure Active Directory only authentication is enabled" when testing the SQL connection	The SQL server does not accept usernames and passwords	Edit `ls_sql_orders`, set Authentication type to **System-assigned managed identity**, then test again
"Login failed for user '<token-identified principal>'" or "Principal could not be found"	The `CREATE USER` name does not match your Data Factory name	Rerun the four SQL lines with the exact Data Factory name
The Container box is empty when browsing for the file	The container or file does not exist, or the connection is broken	Create container `orders` and upload the CSV, then test `ls_blob_orders`. As a shortcut type `orders` and the file name by hand
The table name dropdown is empty	The SQL linked service cannot connect yet	Fix the linked service first, or tick **Enter manually** and type `dbo` and `orders_clean`
SQL connection times out the first time	The serverless database was paused and is waking up	Wait about one minute and test again
Firewall error on the SQL server	Azure services are not allowed in	SQL server → **Networking** → tick **Allow Azure services and resources to access this server** → Save
Aggregate fails or shows an `order_id` error	The matching condition included `order_id`	Set the matching condition to `name != 'order_id'`
Data preview shows 7 rows after the Filter	An empty customer shows as blank text, not null	Use the full filter that also checks `trim(customer_id) != ''`
Data flow debug takes forever to start	The helper cluster is warming up	Wait up to 5 minutes. Do not toggle it off and on
Pipeline fails on duplicate key	The table was not emptied before the load	Add the `TRUNCATE TABLE dbo.orders_clean` pre-SQL script in the sink

What you see	What it means	Fix
Source has no columns	Schema was not imported	With debug on, open the source **Projection** tab and click **Detect data types**

Keep costs low and clean up

- Turn **Data flow debug** off when you stop working. The helper computers bill while they stand ready.
- The serverless SQL database pauses itself after 1 hour of no use.
- When the whole lab is done, open the Azure portal → **Resource groups** → `rg-etl-training` → **Delete resource group**. Type the name to confirm. Every resource you created is removed and billing stops.

Final checklist

- [ ] Resource group `rg-etl-training` exists
- [ ] Storage account has container `orders` with `orders_2026-10-24.csv` inside
- [ ] SQL database `training` has an empty table `dbo.orders_clean`
- [ ] Data Factory has permission in SQL (the four SQL lines ran successfully)
- [ ] Linked services `ls_blob_orders` and `ls_sql_orders` both pass **Test connection**
- [ ] Datasets `ds_orders_csv` and `ds_orders_sql` exist
- [ ] Data flow `df_clean_orders` shows 7, 7, 5, 4, 4 rows in the data preview
- [ ] Pipeline `pl_orders_etl` ran with status **Succeeded**
- [ ] `SELECT * FROM dbo.orders_clean` returns exactly 4 rows
- [ ] Everything is published, and debug is turned off